

Wisdom & Chance TCG

Mobile Sync Guide & Feature Roadmap

API Version 2.3.0 | March 2026

For the Mobile Development Team

Base URL: <https://wisdom-and-chance.replit.app>
WebSocket: <wss://wisdom-and-chance.replit.app/ws>

Table of Contents

1. Current Feature Inventory
2. Gap Analysis vs Major CCGs
3. Server-Sync Architecture Guide
4. API Contracts & Versioning
5. Economy System (Implemented)
6. Upcoming Features & Roadmap
7. WebSocket Contract
8. Data Flow Diagrams
9. Mobile-Specific Recommendations
10. Appendix: Quick Reference

1. Current Feature Inventory

1.1 Core Game Systems

Feature	Status	Platform	Notes
200 Unit Cards (5 elements)	Live	Web + API	Power 1-10, traits, buffs/ debuffs
10 Commanders	Live	Web + API	4 ability categories
Deck Builder	Live	Web + API	40 cards, power distribution rules
Practice vs AI	Live	Web + API	Easy/Medium/Hard difficulty
Card Database	Live	Web + API	Browse all cards with filters
Card Art (AI)	Live	Admin	Gemini 2.5 Flash image generation
Rarity System	Live	Web + API	Common/Rare/Epic/ Legendary from power

1.2 Multiplayer & Social

Feature	Status	Platform	Notes
Real-time PvP	Live	Web + WS	Server-authoritative engine
Game Rooms	Live	Web + API + WS	Public/private, spectator support
Matchmaking	Live	Web + WS	ELO-based queue
In-game Chat	Live	Web + WS	Room + game chat
Spectator Mode	Live	Web + WS	Watch live games
Friend System	Live	Web + API	Requests, online status
Emotes	Live	Web + WS	In-game reactions

1.3 Progression & Engagement

Feature	Status	Platform	Notes
Achievements	Live	Web + API	5 categories, XP + gold rewards
Daily Challenges	Live	Web + API	Rotating daily goals
Leaderboard	Live	Web + API	ELO tiers: Bronze to Master
Player Stats	Live	Web + API	Level, XP, streaks, favorites
Economy System	Built	Web + API	Feature-flagged, ready to enable
Collection System	Built	Web + API	Card ownership, starter grants

1.4 Infrastructure

Feature	Status	Notes
Unified Auth	Live	Session cookies (web) + JWT Bearer (mobile)
Mobile Auth	Live	POST /api/mobile/auth/login with JWT (7-day)
Feature Flags	Live	Server-controlled, 15+ flags
API Docs	Live	GET /api/docs — full JSON reference
Admin Sync	Live	GET /api/admin/sync (code-protected)
PWA Support	Live	Manifest, service worker, icons
Database Backups	Live	Admin export + pg_dump
Health Check	Live	GET /api/health

2. Gap Analysis vs Major CCGs

Comparison against Hearthstone, MTG Arena, and Master Duel. Features are categorized by implementation priority.

2.1 Tier 1 — Critical (Core Progression Loop)

Feature	Hearthstone	MTG Arena	W&C Status	Priority
Card Rarity	Yes	Yes	DONE	—
Card Collection	Yes	Yes	DONE (flagged)	—
In-game Currency	Gold/Dust	Gold/Gems	DONE (flagged)	—
Pack Opening	Yes (animated)	Yes	Basic (no animation)	Next
Crafting/Disenchant	Yes	Wildcards	DONE (flagged)	—
Shop / Store	Yes	Yes	Not built	Next
Starter Decks	Yes	Yes	Starter collection done	—

2.2 Tier 2 — High (Daily Engagement)

Feature	Hearthstone	MTG Arena	W&C Status	Priority
Seasons / Ranked	Monthly	Monthly	Schema planned	Soon
Battle Pass	Yes (Tavern)	Mastery Pass	Schema planned	Soon
Daily Login	Quests	Daily rewards	Not built	Medium
Match History	Recent games	Yes	Not built	Medium
Tutorials	Interactive	Color challenge	Basic tutorial	Low

2.3 Tier 3 — Medium (Collectibility & Monetization)

Feature	Hearthstone	MTG Arena	W&C Status	Priority
Card Variants	Golden/Diamond	Styles/Alt art	Not built	Medium
Cosmetics	Hero skins	Pets/Sleeves	Not built	Medium
Premium Currency	Runestones	Gems	Gems reserved	Medium
Tournaments	Battlegrounds	Events	Not built	Low
Draft/Arena	Arena mode	Draft/Sealed	Not built	Low

2.4 Tier 4 — Nice to Have

Feature	Status	Notes
Sound/Music	Not built	Audio engine + asset pipeline
Card Animations	Not built	Play/attack/death VFX
Guilds/Alliances	Not built	Social feature
Limited-time Modes	Not built	Seasonal game variants
Replays	Not built	Match recording/playback

3. Server-Sync Architecture Guide

3.1 Thin Client Principles

The mobile app should be a thin client that relies on the server as the single source of truth for all game state and data.

- Never trust client-side calculations for currency, collection, or game state
- All mutations go through server API — server validates and returns canonical state
- Card data, commanders, and game constants come from the server (cache locally)
- Feature flags control what UI to show — always respect GET /api/config
- Game logic is server-authoritative — clients send actions, server resolves them

3.2 Feature Flags

Every major feature is gated by a boolean flag returned from GET /api/config. The mobile app **MUST** check these before rendering UI sections.

```
GET /api/config !' response.features
{
  "economy_enabled": false,    // Shop, currencies, collection
  "practice_mode": true,      // Practice vs AI
  "multiplayer": true,        // PvP, rooms, matchmaking
  "friends_system": true,     // Friend requests, status
  "daily_challenges": true,   // Daily goals
  "achievements": true,      // Achievement tracking
  "leaderboard": true,       // ELO rankings
  "spectator_mode": true,    // Watch live games
  "ranked_seasons": false,   // Seasonal ranking
  "battle_pass": false,     // Season pass rewards
  "shop_enabled": false,    // In-game shop
  "emotes": false           // In-game reactions
}
```

3.3 Versioned API

The server returns apiVersion in GET /api/config. The mobile app should:

- Store its own minimum supported API version
- Compare against server's apiVersion on launch
- If server version < mobile minimum, show 'server update pending' notice
- If mobile version < server's minClientVersion, force app update prompt

3.4 Authentication Flow

1. Mobile calls POST /api/mobile/auth/login { email, provider }
2. Server returns { token: "JWT...", user: {...} }
3. Store JWT securely (Keychain/Keystore)
4. Include "Authorization: Bearer <token>" on all requests
5. Token expires in 7 days
6. Before expiry: POST /api/mobile/auth/refresh !' new token
7. On 401: redirect to login screen

3.5 Data Sync Strategy

Recommended caching and sync approach for mobile:

Data Type	Cache Duration	Sync Trigger	Strategy
Cards/Commanders	24 hours	App launch	Cache aggressively, rarely changes
Config/Flags	5 minutes	App launch + resume	Short cache, affects UI
Collection	On mutation	Pack open, craft, disenchant	Invalidate on write
Currencies	On mutation	Any economy action	Invalidate on write
Decks	On mutation	Create/edit/delete	Invalidate on write
Friends	2 minutes	Friend list view	Moderate cache
Leaderboard	5 minutes	Leaderboard view	Moderate cache
Game State	Never cache	WebSocket real-time	Always live via WS

4. API Contracts & Versioning

4.1 Base Configuration

```
Base URL: https://wisdom-and-chance.replit.app
API Prefix: /api
Auth Header: Authorization: Bearer <jwt_token>
Content-Type: application/json
Current Version: 2.3.0
```

4.2 Authentication Endpoints

Method	Path	Auth	Description
POST	/api/mobile/auth/login	No	Get JWT token (7-day expiry)
POST	/api/mobile/auth/refresh	Yes	Refresh JWT token
GET	/api/mobile/auth/me	Yes	Get current user info
GET	/api/auth/user	Yes	Get auth user (both web/mobile)

4.3 Public Endpoints (No Auth)

Method	Path	Description
GET	/api/health	Health check + DB status
GET	/api/config	Feature flags, version, maintenance
GET	/api/cards	All 200 cards with rarity
GET	/api/cards/:id	Single card by ID
GET	/api/cards/element/:el	Cards by element
GET	/api/commanders	All 10 commanders
GET	/api/commanders/:id	Single commander
GET	/api/achievements	All achievements
GET	/api/daily-challenges	Today's challenges
GET	/api/leaderboard	Top 100 by ELO
GET	/api/rooms	Public waiting rooms
GET	/api/docs	Full API reference (JSON)

4.4 Protected Endpoints (Auth Required)

Method	Path	Description
PATCH	/api/user/profile	Update profile info
GET	/api/user-decks	List user's saved decks
POST	/api/user-decks	Create deck (40 cards, validated)
PATCH	/api/user-decks/:id	Update deck
DELETE	/api/user-decks/:id	Delete deck
GET	/api/player-stats	Player stats (level, XP, streaks)
GET	/api/player-achievements	User's achievement progress
GET	/api/player-challenges	User's challenge progress
POST	/api/player-challenges/:id/claim	Claim challenge reward

4.5 Multiplayer & Room Endpoints

Method	Path	Description
POST	/api/rooms	Create game room
POST	/api/rooms/:id/join	Join a room
POST	/api/rooms/:id/leave	Leave a room
POST	/api/rooms/:id/ready	Toggle ready status
POST	/api/rooms/:id/start	Start game (host only)
POST	/api/rooms/:id/spectate	Join as spectator
DELETE	/api/rooms/:id/spectate	Leave spectating
GET	/api/rooms/:id/messages	Get room chat history
POST	/api/rooms/:id/messages	Send room chat message
GET	/api/live-matches	Active matches for spectating

4.6 Friend System Endpoints

Method	Path	Description
GET	/api/friends	List accepted friends
GET	/api/friend-requests	List pending requests
POST	/api/friend-requests	Send friend request
POST	/api/friend-requests/:id/accept	Accept request
POST	/api/friend-requests/:id/decline	Decline request

5. Economy System (Implemented)

The economy system is fully built and feature-flagged behind 'economy_enabled'. When enabled, it adds currencies, card ownership, pack opening, crafting, and disenchanting.

5.1 Rarity System

Rarity	Power Range	Pack Weight	Craft Cost (Dust)	Disenchant (Dust)
Common	1-3	60%	40	5
Rare	4-6	25%	100	20
Epic	7-8	10%	400	100
Legendary	9-10	5%	1,600	400

5.2 Currency Constants

Constant	Value	Notes
Starter Gold	500	Granted on first login
Pack Cost	100 gold	5 cards per pack
Match Win	30 gold	Auto-granted via WebSocket
Match Loss	10 gold	Auto-granted via WebSocket
Match Draw	15 gold	Auto-granted via WebSocket
Forfeit Win	15 gold	Opponent disconnected
Daily Challenge	25 gold	On claim (economy_enabled)
Achievement	50 gold	One-time per achievement

5.3 Economy Endpoints

Method	Path	Description
GET	/api/currencies	Get gold/gems/dust balances
GET	/api/collection	Get owned cards with quantities
POST	/api/packs/open	Buy & open pack (100 gold)
POST	/api/cards/craft	Craft card with dust
POST	/api/cards/disenchant	Disenchant for dust
POST	/api/collection/starter	Claim starter (idempotent)
POST	/api/achievements/:id/claim	Claim achievement gold

5.4 Starter Collection

On first login (web or mobile), players receive:

- 500 Gold
- 2 copies of every power 1-5 card (Common + Rare) across all 5 elements
- This is tracked by a 'starterClaimed' flag — calling /api/collection/starter is idempotent

5.5 Pack Opening Response

```
POST /api/packs/open
Response: {
  "cards": [
    { "cardId": "card-fire-3-2", "rarity": "Common", "isNew": true },
    { "cardId": "card-water-7-1", "rarity": "Epic", "isNew": false },
    ...
  ],
  "costGold": 100,
  "remainingGold": 400
}
```

5.6 Deck Ownership Validation

When economy is enabled, deck create/update validates that the player owns sufficient copies of each card. The server rejects decks containing cards the player doesn't own.

6. Upcoming Features & Roadmap

6.1 Next Up: Shop & Pack Opening Experience

Feature flag: shop_enabled, pack_opening_enabled

- Visual shop page with pack types and pricing
- Pack opening animation (card flip reveal with rarity glow)
- Multiple pack types: Basic (100g), Premium (250g), Element (150g)
- Purchase history and pack opening log
- API: GET /api/shop/packs, POST /api/shop/buy-pack

6.2 Ranked Seasons

Feature flag: ranked_seasons

- Monthly seasons with themes (e.g., 'Season 1: Flames of Dawn')
- ELO soft-reset at season end (50% toward 1000)
- Season-end rewards based on final tier
- Season countdown timer synced with server time
- API: GET /api/seasons/current, GET /api/seasons/:id/rewards

6.3 Battle Pass

Feature flag: battle_pass

- 30-tier reward track per season
- Free track: Gold, basic packs, common/rare cards
- Premium track: More gold, premium packs, exclusive cards
- XP from matches, challenges, achievements drives progression
- API: GET /api/battle-pass, POST /api/battle-pass/claim/:tier

6.4 Daily Login Rewards

7-day reward cycle with escalating rewards:

- Day 1: 10 Gold !' Day 7: Premium Pack + 50 Gold
- Streak counter and calendar UI
- API: POST /api/daily-login/claim

6.5 Card Variants & Cosmetics

Future monetization layer:

- Foil, animated, and alt-art card variants
- Same stats, different visuals
- Obtainable through packs, crafting, or shop
- Premium currency (Gems) for direct purchase

6.6 Implementation Priority Order

Phase	Features	Estimated Timeline	Dependencies
Phase 1	Shop + Pack Animation	Next sprint	Economy system (done)
Phase 2	Seasons + Battle Pass	Following sprint	Shop (phase 1)
Phase 3	Daily Login + Variants	After phase 2	Seasons (phase 2)
Phase 4	Tournaments + Draft	Future	Stable economy

7. WebSocket Contract

7.1 Connection

```
// Mobile WebSocket connection
const ws = new WebSocket(
  "wss://wisdom-and-chance.replit.app/ws?token=<jwt_token>"
);

// Web uses session cookies automatically
// Mobile MUST pass JWT via query parameter
```

7.2 Client !' Server Events

Event	Payload	Description
join_room	{ roomId }	Subscribe to room updates
leave_room	{ roomId }	Unsubscribe from room
join_game	{ gameId }	Subscribe to game state
leave_game	{ gameId }	Unsubscribe from game
room_message	{ roomId, message }	Send chat in room
game_message	{ gameId, message }	Send chat in game
game_action	{ gameId, action }	Send game action
player_ready	{ roomId, ready, deckId? }	Toggle ready + select deck
game_start	{ roomId }	Request game start (host)
room_update	{ roomId }	Request room state refresh
ping	{ }	Heartbeat keepalive

7.3 Server !' Client Events

Event	Payload	Description
auth_success	{ userId, displayName }	Auth confirmed, ready to send
auth_error	{ message }	Auth failed, will disconnect
room_update	Room state object	Room state changed
player_joined	{ roomId, userId }	Player joined room
player_left	{ roomId, userId }	Player left room
player_ready_update	{ roomId, userId, ready, deckId }	Ready status changed
game_start	{ roomId, gameId }	Game started — navigate to game
game_state	{ ...sanitized state }	Game state (per-player view)
game_action	Game action object	Game update from opponent
game_over	{ winner, reason, goldReward? }	Game ended + rewards
chat_message	{ roomId, senderId, message }	Room chat message
game_message	{ userId, displayName, message }	In-game chat message
friend_request	{ requestId, senderId }	Incoming friend request
friend_request_accepted	{ requestId, friendId }	Request accepted
presence_update	{ userId, status }	Friend online/offline
spectator_joined	{ roomId, userId }	Spectator joined
spectator_left	{ roomId, userId }	Spectator left
error	{ message }	Server error

7.4 Game Action Types

```
game_action payload:
{ gameId: "uuid", action: {
  type: "draw" | "deploy" | "end_turn" | "use_ability" | "forfeit",
  ...action-specific fields
}}

draw:      { type: "draw" }
deploy:    { type: "deploy", cardId: "card-fire-3-2", slot: 0-3 }
use_ability: { type: "use_ability" }
end_turn:   { type: "end_turn" }
forfeit:    { type: "forfeit" }
```

7.5 Reconnection Handling

The server allows 60-second reconnection window for disconnected players:

- If a player disconnects during a game, their opponent gets a 60s countdown
- Reconnecting within 60s resumes the game — server sends full game_state
- After 60s, the disconnected player auto-forfeits
- Mobile should implement exponential backoff reconnect (1s, 2s, 4s, 8s, max 30s)

8. Data Flow Diagrams

8.1 Economy Transaction Flow

```
% % % % % % % % % % POST /api/packs/open % % % % % % % % % %
% Mobile % % % % % % % % % % % % % % % % % % % % % % % % Server %
% Client % % % % % % % % % % % % % % % % % % % % % % % %
% % % % % % % % % % { cards[], remainingGold } % % % % % % % % % %
% % % % % % % % % %<% % % % % % % % % %
% % % % % % % % % %
Check gold Deduct Add cards
>= 100 gold to collection
(WHERE) (atomic) (upsert)
% % % % % % % % % %<% % % % % % % % % %
% % % % % % % % % %
% DB %
% % % % % % % % % %
```

8.2 Match Reward Flow

% % % % % % % % %	game_action	% % % % % % % % %	game_over	% % % % % % % % %
% Player A % % % % % % % % %	% % % % % % % % %	% Server % % % % % % % % %	% % % % % % % % %	% Player B % % % % % % % % %
% % % % % % % % %	% Engine %	% % % % % % % % %	% % % % % % % % %	
	% % % % % , % % % % %			
	%			
	% % % % % % % % % % % % % % < % % % % % % % % % %			
	%	%	%	
	Record game	Grant gold	Grant gold	
	to database	to winner	to loser	
	%	(30 gold)	(10 gold)	
	%	%	%	
	% % % % % % % % % % % % % % < % % % % % % % % % %			
	%			
	% % % % % % % % % % % % % %			
	% DB %			
	% % % % % % % % % % % % %			

Note: Gold only granted when `economy_enabled = true`

8.3 Authentication Flow

%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%		
%	Mobile	%	POST /api/mobile/									%	Server	%							
%	App	%	auth/login									%		%							
%		%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%°		
%		%	{ email, provider }									%		%							
%		%										%		%							
%		%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%	%		
%		%	{ token, user }									%		%							
%		%										%		%							
%	Store	%	Bearer <token>									%	Verify	%							
%	in	%	on all requests									%	JWT on	%							

%

%

% request %

% % % % % % % % % % % %

% % % % % % % % % % % %

Token refresh: POST /api/mobile/auth/refresh

Returns new 7-day token. Call before expiry.

8.4 Season Progression (Planned)

```
Match Win/Challenge Complete/Achievement Unlock
    %
    %¼
    Grant XP to player
    %
    %¼
    Check Battle Pass tier
    (currentXp >= tierThreshold?)
    %
    % % % % % %4% % % % %
    % Yes      % No
    %¼         %¼
    Unlock tier   Continue
    rewards      playing
    (notify)
    %
    %¼
    Season End !' Soft-reset ELO
                !' Grant tier rewards
                !' Archive to history
```


9. Mobile-Specific Recommendations

9.1 Offline Caching Strategy

- Cache card data and commanders in local storage (SQLite/AsyncStorage)
- Cache user's decks for offline viewing (not editing)
- Cache leaderboard and friend list with timestamps
- Show cached data immediately, refresh in background
- Never cache game state — always require live connection for play
- Cache feature flags for 5 minutes max

9.2 Push Notification Hooks

The server can be extended with push notification triggers for:

- Friend request received
- Challenge/match invitation
- Daily challenge refresh (midnight UTC)
- Season ending soon (24h, 1h warnings)
- Battle pass tier unlocked
- Achievement completed

Mobile should register device token via future endpoint: POST /api/push/register

9.3 Reconnection Handling

- Implement WebSocket heartbeat (ping every 30s)
- Exponential backoff on disconnect: 1s !' 2s !' 4s !' 8s !' max 30s
- On reconnect during active game: server auto-sends full game_state
- 60-second window before auto-forfeit — show reconnection countdown to user
- On reconnect outside game: re-fetch /api/config, re-establish presence

9.4 Image Handling

- Card images available at: GET /api/cards/:id/image
- Commander images at: GET /api/commanders/:id/image
- Cache images aggressively — they rarely change
- Use placeholder/skeleton images while loading
- Consider progressive image loading for card database

9.5 Error Handling

Standard error response format:

```
{
  "message": "Human-readable error description",
  "error": "Error type identifier" // optional
}
```

Common status codes:

- 400 — Validation error (check message for details)
- 401 — Token expired or invalid (redirect to login)
- 403 — Not authorized for this resource
- 404 — Resource not found
- 429 — Rate limited (back off and retry)
- 500 — Server error (show generic error, retry)
- 503 — Maintenance mode (check /api/config)

9.6 Performance Tips

- Batch API calls on app launch: /api/config + /api/cards + /api/auth/user
- Use ETag/If-None-Match headers for card data caching
- Paginate leaderboard requests (server returns top 100)
- Debounce deck save calls (500ms delay after last edit)
- Pre-fetch next game state on opponent's turn

10. Appendix: Quick Reference

10.1 Card ID Format

```
card-{element}-{power}-{variant}
Examples: card-fire-3-2, card-water-7-1, card-earth-10-0
```

10.2 Element List

```
Fire | Water | Earth | Air | Nature
(5 elements, 40 cards each = 200 total)
```

10.3 Deck Validation Rules

Rule	Value
Total cards	Exactly 40
Cards per power rank	4 (ranks 1-10)
Max copies per card	3
Commander required	Yes (1 per deck)
Ownership check	When economy_enabled=true

10.4 ELO Rating Tiers

Tier	Rating Range
Bronze	< 800
Silver	800 – 999
Gold	1000 – 1199
Platinum	1200 – 1399
Diamond	1400 – 1599
Master	1600+

10.5 Game Constants

```
Starting HP: 40
Cards per hand: 5 (initial draw)
Deployment slots: 4 per side
Turn phases: Draw !' Deploy !' Combat !' Calculate !' End
Max power per card: 10
Buff/Debuff colors: Red, Blue, Green, Yellow, Purple
```

10.6 Key Server Files (For Reference)

File	Purpose
server/apiDocs.ts	Full API documentation (JSON)
server/routes.ts	All REST endpoints
server/websocket.ts	WebSocket event handling
server/gameEngine.ts	Server-authoritative game engine
server/economyService.ts	Economy helpers (grantGold, etc.)
server/multiplayerRoutes.ts	Multiplayer & social endpoints
server/mobileAuth.ts	Mobile JWT auth
shared/schema.ts	DB schema + types
shared/models/economy.ts	Economy tables + constants

10.7 Admin Endpoints (For Debugging)

Note: Admin endpoints require admin authentication.
Contact the project owner for access codes.

Method	Path	Description
GET	/api/admin/sync	Full data dump (code-protected)
GET	/api/admin/feature-flags	View all feature flags
PATCH	/api/admin/feature-flags/:key	Toggle a feature flag
GET	/api/admin/database-export	Export all DB tables as JSON